

Федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Институт заочного образования

09.03.01 "Информатика и вычислительная техника"
профиль "Программное обеспечение средств
вычислительной техники и автоматизированных систем"

Кафедра прикладной математики и кибернетики

Курсовая работа по дисциплине Вычислительная математика

Нахождение количества теплоты

Вариант 9

Выполнил:

студент гр. ЗП-31

«__» _____ 2025 г.

_____/Ушакова Е.А./
ФИО студента

Проверил

доцент каф. ПМиК

«__» _____ 2025 г.

_____/Галкина М.Ю./

Оценка _____

Новосибирск 2025 г.

СОДЕРЖАНИЕ

Оглавление

1. УСЛОВИЕ ЗАДАЧИ	3
2. РЕЗУЛЬТАТЫ АНАЛИТИЧЕСКИХ РАСЧЕТОВ	5
3. ОПИСАНИЕ ФОРМУЛЫ И ИСПОЛЬЗУЕМЫХ МЕТОДОВ	6
3.1 Метод хорд	6
3.2 Решение дифференциального уравнения методом Рунге-Кутты четвертого порядка ..	6
3.3 Линейная интерполяция	6
3.4 Метод трапеции	7
4. ИСХОДНЫЙ КОД ПРОГРАММЫ	8
5. РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ	12

1. УСЛОВИЕ ЗАДАЧИ

Напряжение в электрической цепи описывается дифференциальным уравнением с начальным условием.

1. Найти аналитически интервал изоляции положительного корня заданного нелинейного уравнения, вычислив производную левой части уравнения и составив таблицу знаков левой части уравнения на всей числовой оси.
2. Написать программу, которая:
 - а) находит k – наименьший положительный корень заданного нелинейного уравнения из найденного в пункте 1 интервала изоляции с точностью 0.001 методом: деления пополам (если Ваше имя начинается на гласную букву), хорд (если Ваше имя начинается на согласную букву);
 - б) решает дифференциальное уравнение методом Рунге-Кутты четвертого порядка с точностью 10^{-4} на интервале $[0;2]$ (для достижения заданной точности использовать метод двойного пересчета, начальный шаг решения взять равным 1);
 - в) с помощью линейной интерполяции по найденному в пункте б) решению дифференциального уравнения находит приближенные значения функции в точках $x = 0, 0.1, 0.2, \dots, 1.9, 2, i = 0, 1, \dots, 20$;
 - г) определяет количество теплоты $Q = \int_0^2 y^2 dt$, выделяющееся на единичном сопротивлении за 2 единицы времени, методом: Симпсона (если Ваша фамилия начинается на согласную букву), трапеций (если Ваша фамилия начинается на гласную букву) с шагом 0.1.
3. Программа должна выводить:
 - а) найденное приближенное значение k и количество итераций, которое потребовалось для достижения заданной точности;
 - б) решение дифференциального уравнения на интервале $[0;2]$ с заданной точностью (выводить следует в 2 столбика: значение x и соответствующее ему значение y);
 - в) результаты линейной интерполяции в точках $x = 0, 0.1, 0.2, \dots, 1.9, 2, i = 0, 1, \dots, 20$; (выводить следует в 2 столбика: значение x_i и соответствующее ему значение y_i);
 - г) количество теплоты Q .

Вариант 9:

$$\begin{cases} y' = 6 \sin x - (3 + x)y \\ y(0) = k \end{cases}$$

Где k - наименьший положительный корень уравнения $3x^4 + 4x^3 - 12x^2 - 40 = 0$

2.РЕЗУЛЬТАТЫ АНАЛИТИЧЕСКИХ РАСЧЕТОВ

Найдем интервал изоляции положительного корня заданного уравнения

$$3x^4 + 4x^3 - 12x^2 - 40 = 0$$

Для этого найдем производную функции

$$f'(x) = 12x^3 + 12x^2 - 24x$$

Найдем критические точки:

$$12x^3 + 12x^2 - 24x = 0$$

$$12x * (x^2 + x - 2) = 0$$

$$x^2 + x - 2 = 0 \text{ или } x = 0$$

$$D = 1 - 4 * (-2) = 9$$

$$x_{1,2} = \frac{-1 \pm \sqrt{9}}{2} \quad x_1 = 1, \quad x_2 = -2$$

Составим таблицу знаков функции

x	$-\infty$	-2	0	1	$+\infty$
f	$+$	$-$	$-$	$-$	$+$

Уравнение имеет два корня на интервалах $(-\infty; -2)$ и $(1; +\infty)$

Уменьшим промежутки

x	-4	-3	2	3
f	$+$	$-$	$-$	$+$

Итак, уравнение имеет два корня $x_1 (-4; -3)$ $x_2 (2; 3)$

Интервал изоляции наименьшего положительного корня $(2; 3)$.

3. ОПИСАНИЕ ФОРМУЛЫ И ИСПОЛЬЗУЕМЫХ МЕТОДОВ

3.1 Метод хорд

В методе хорд интервал делится на неравные части в отношении $f(a) : f(b)$ хордой, соединяющей точки $(a, f(a))$ и $(b, f(b))$. Пусть имеется интервал $[a, b]$, на котором функция $f(x)$ меняет свой знак, т.е. $f(a) * f(b) < 0$

Для точки пересечения хорды с осью Ox ($x = c, y = 0$) получаем:

$$x_0 = a - \frac{b - a}{f(b) - f(a)} * f(a)$$

Далее из отрезков $[a, c]$ и $[c, b]$ выбираем тот, на котором функция меняет знак. Следующая итерация состоит в определении нового приближения как точки пересечения новой хорды с осью Ox и т.д. Повторяем процесс до тех пор, пока не будет достигнута заданная точность.

3.2 Решение дифференциального уравнения методом Рунге-Кутты четвертого порядка

На практике чаще всего используют метод Рунге-Кутты четвертого порядка:

$$\begin{aligned} k_1 &= f(x_i, y_i) \\ k_2 &= f\left(x_i + \frac{h}{2}, y_i + \frac{h}{2} * k_1\right) \\ k_3 &= f\left(x_i + \frac{h}{2}, y_i + \frac{h}{2} * k_2\right) \\ k_4 &= f(x_i + h, y_i + h * k_3) \\ y_{i+1} &= y_i + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4) \end{aligned}$$

Погрешность этого метода равна $C^5 \cdot h^4$.

3.3 Линейная интерполяция

Формула линейной интерполяции:

$$f(x) \approx y_i + q \Delta y_i, \quad q = \frac{x - x_i}{h} \quad \text{при } x \in [x_i; x_{i+1}]$$

Геометрически линейная интерполяция означает замену графика функции на отрезке $[x_i, x_{i+1}]$ хордой, соединяющей точки (x_i, f_i) и (x_{i+1}, f_{i+1})

3.4 Метод трапеции

Пусть известны значения функции $y = f(x)$ в точках $x_0, x_1, \dots, x_n$: $y_i = f(x_i)$.

Необходимо найти значение $\int_a^b f(x) dx$. Основная идея численного интегрирования заключается в том, чтобы заменить функцию $f(x)$ на

интерполирующую ее функцию $y(x)$ (которую можно проинтегрировать)

Соединяя каждые два узла прямой (т.е. применяя линейную интерполяцию)

получим, что площадь под кривой приближенно равна сумме площадей

трапеций. Пусть $x_i - x_{i-1} = \text{const} = h$, при $i = 1, 2, \dots, n$.

Тогда

$$\int_{a=x_0}^{b=x_n} f(x) dx = h \left(\frac{y_0 + y_n}{2} + y_1 + y_2 + \dots + y_{n-1} \right)$$

4.ИСХОДНЫЙ КОД ПРОГРАММЫ

```
#include <stdlib.h>
#include <stdio.h>
#include <math.h>

int iteration;

double xr [21] = {0};
double yr [21] = {0};
double h;

double xi [21] = {0};
double yi [21] = {0};

double func (double x) {
    return 3* pow(x, 4) + 4 * pow(x, 3) - 12 * pow(x, 2) - 40;
}

double chordMethod(double a, double b, double epsilon) {
    double fa = func(a);
    double fb = func(b);
    double x, fx;
    iteration = 0;

    do {
        x = a - fa * (b - a) / (fb - fa);
        fx = func(x);

        if (fa * fx < 0) {
            b = x;
            fb = fx;
        } else {
            a = x;
            fa = fx;
        }

        iteration++;

    } while (fabs(fx) > epsilon);

    return x;
}

double diff(double x, double y) {
    return 6 * sin(x) - (3 + x) * y;
}

double runge_kutta_step(double x, double y, double h) {
    double k1, k2, k3, k4;
```



```

    k1 = diff(x, y);
    k2 = diff(x + h/2, y + h*k1/2);
    k3 = diff(x + h/2, y + h*k2/2);
    k4 = diff(x + h, y + h*k3);

    return y + h * (k1 + 2*k2 + 2*k3 + k4) / 6;
}

int step_count;
double solveODE(double x0, double y0, double x_end, double epsilon) {
    double x = x0;
    double y = y0;
    double h = 1.0;
    step_count = 0;

    while (x < x_end) {

        if (x + h > x_end) {
            h = x_end - x;
        }

        double y1, y2, error;

        do {
            y1 = runge_kutta_step(x, y, h);

            double y_temp = runge_kutta_step(x, y, h/2);
            y2 = runge_kutta_step(x + h/2, y_temp, h/2);

            error = fabs(y1 - y2) / 15.0;

            if (error > epsilon) {
                h /= 2;
            }
        } while (error > epsilon);

        double y_next = y2;

        y = y_next;
        x += h;

        step_count++;
        xr[step_count] = x;
        yr[step_count] = y;
    }

    return h;
}

int findLower(double x) {

```

```

    int i = 0;
    for (i = 1; i < step_count; i++) {
        if (xr[i] >= x) break;
    }
    i--;
    return i;
}

int findHigher(double x) {
    return (findLower(x) + 1);
}

double findQ (double x) {
    double xi = xr[findLower(x)];
    return (x - xi) / h;
}

int main () {
    double a = 2.0;
    double b = 3.0;
    double epsilon = 0.001;

    double k = chordMethod(a, b, epsilon);
    double x0 = 0.0;
    double x_end = 2.0;

    printf("\nНайден корень: %.4f за %d итераций\n", k, iteration);
    printf("Решение ОДУ методом Рунге-Кутта 4-го порядка\n");
    printf("|    x    | y_прибл | \n");

    xr[0] = xi[0] = x0;
    yr[0] = yi[0] = k;

    h = solveODE(xi[0], yi[0], x_end, 0.0001);
    for (int i = 0; i <= step_count; i++) {
        printf("| %7.4f | %9.4f | \n", xr[i], yr[i]);
    }

    printf("Результаты линейной интерполяции\n");
    printf("|    x    | y_прибл | \n");
    double q = findQ(xi[0]);
    for (int i = 0; i < 21; i++) {
        xi[i] = 0.1 * i;
        q = findQ(xi[i]);
        yi[i] = yr[findLower(xi[i])] + q * (yr[findHigher(xi[i])] -
yr[findLower(xi[i])]);
        printf("| %7.1f | %9.4f | \n", xi[i], yi[i]);
    }
}

```

```
double qt = ((yi[0] * yi[0] + yi[20] * yi[20]) / 2);

for (int i = 1; i < 20; i++) {
    qt += yi[i] * yi[i];
}
qt = qt * 0.1;

printf("Найденное количество теплоты: %.4f\n", qt);

return 0;
}
```

5.РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ

Найден корень: 2.0779 за 14 итераций
Решение ОДУ методом Рунге-Кутты 4-го порядка

x	y_прибл
0.0000	2.0779
0.1250	1.4583
0.2500	1.0967
0.3750	0.9158
0.5000	0.8562
0.6250	0.8730
0.7500	0.9334
0.8750	1.0139
1.0000	1.0981
1.1250	1.1751
1.2500	1.2379
1.3750	1.2823
1.5000	1.3062
1.6250	1.3090
1.7500	1.2908
1.8750	1.2526
2.0000	1.1958

Результаты линейной интерполяции

x	y_прибл
0.0	2.0779
0.1	1.5822
0.2	1.2413
0.3	1.0243
0.4	0.9039
0.5	0.8562
0.6	0.8697
0.7	0.9093
0.8	0.9656
0.9	1.0307
1.0	1.0981
1.1	1.1597
1.2	1.2128
1.3	1.2557
1.4	1.2871
1.5	1.3062
1.6	1.3084
1.7	1.2980
1.8	1.2755
1.9	1.2412
2.0	1.1958

Найденное количество теплоты: 2.8621